Mini-compte rendu – Semaine 1

Projet 9 : Deep Hedging (IA avancée) – Groupe 9

Module : Finance des Marchés et ERP

Auteur de la tâche : Mouad
Rôle (S1–S4) : Testeur / Intégration
Tâche S1 : Docker Odoo 17 + PostgreSQL
Date : 11 novembre 2026

---

1 Objectif de la tâche

L'objectif de cette tâche est de mettre en place un **environnement Odoo reproductible** à l'aide de Docker, afin que tous les membres du groupe puissent développer et tester l'extension du module `cours_derives` dans les mêmes conditions. Le livrable attendu est :

- un fichier `docker-compose.yml` fonctionnel (Odoo 17 + PostgreSQL 15) ;
- une capture d'écran prouvant qu'Odoo fonctionne (`docs/docker_odoo.png`) ;
- un commit portant le nom de l'auteur dans le message.

2 Environnement de travail

Élément	Valeur
Système d'exploitation	Linux
Outil de conteneurisation	Docker et Docker Compose
Image ERP	`odoo:17` (version 17.0)
Image base de données	`postgres:15`
Dépôt Git	`deep-hedging-groupe9`
URL d'accès à Odoo	`http://localhost:8069`

Table 1 – Récapitulatif de l'environnement.

3 Étapes réalisées

3.1 Récupération du dépôt

Après acceptation de l'invitation GitHub reçue par e-mail, le dépôt du groupe a été cloné en local :

```
git clone <URL_DU_DEPOT>
cd deep-hedging-groupe9
```

3.2 Préparation de l'arborescence

Le dossier `addons/` a été créé à la racine du dépôt. Il sera monté dans le conteneur Odoo (`/mnt/extra-addons`) et accueillera plus tard le module `cours_derives`, qui n'existe pas encore à ce stade du projet. Un dossier `docs/` contient les preuves (captures) et ce compte rendu.

```
mkdir -p addons docs
touch addons/.gitkeep
```

3.3 Création du fichier `docker-compose.yml`

Le fichier fourni par le chef de projet a été créé à la racine du dépôt (édité avec `nano`) :

```
nano docker-compose.yml
```

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: odoo
      POSTGRES_DB: postgres

  odoo:
    image: odoo:17
    depends_on:
      - db
    ports:
      - "8069:8069"
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    volumes:
      - ./addons:/mnt/extra-addons
```

Explication. Le service `db` lance PostgreSQL 15 avec l'utilisateur `odoo`. Le service `odoo` dépend de `db`, expose le port 8069 sur la machine hôte et se connecte à la base via le nom d'hôte `db` (réseau interne Docker). Le volume `./addons` permettra de charger nos modules personnalisés sans reconstruire l'image.

3.4 Lancement des conteneurs

```
docker compose up -d
```

Les images `odoo:17` et `postgres:15` ont été téléchargées, le réseau `deep-hedging-groupe9_default` a été créé, puis les deux conteneurs ont démarré.

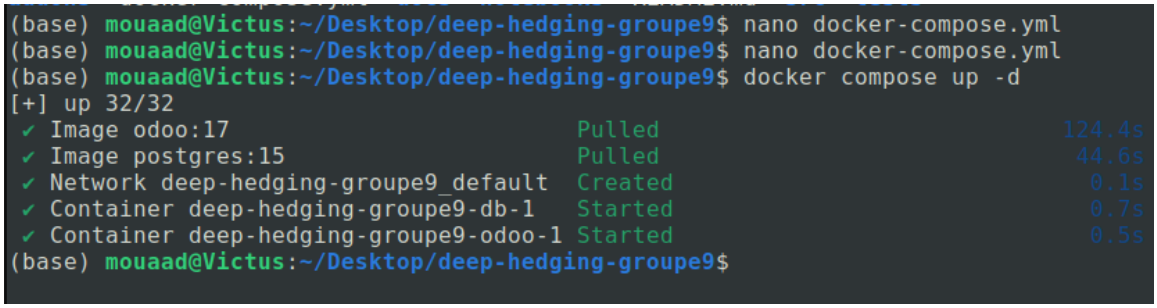


FIGURE 1 – Résultat de `docker compose up -d`.

3.5 Vérification de l'état des conteneurs

```
docker compose ps
```

Les deux services (`db` et `odoo`) sont à l'état *Up*. Odoo est publié sur le port `8069` et PostgreSQL écoute sur le port interne `5432`.

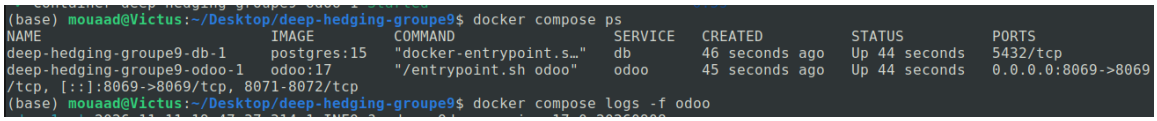


FIGURE 2 – Résultat de `docker compose ps`.

3.6 Analyse des journaux (logs) d'Odoo

```
docker compose logs -f odoo
```

Les journaux confirment le bon fonctionnement :

— version d'Odoo : `17.0` ;
— chemins d'add-ons incluant `/mnt/extra-addons` (notre futur dossier de modules) ;
— connexion à la base PostgreSQL via `odoo@db:5432` ;
— service HTTP (`werkzeug`) démarré sur le port `8069`.

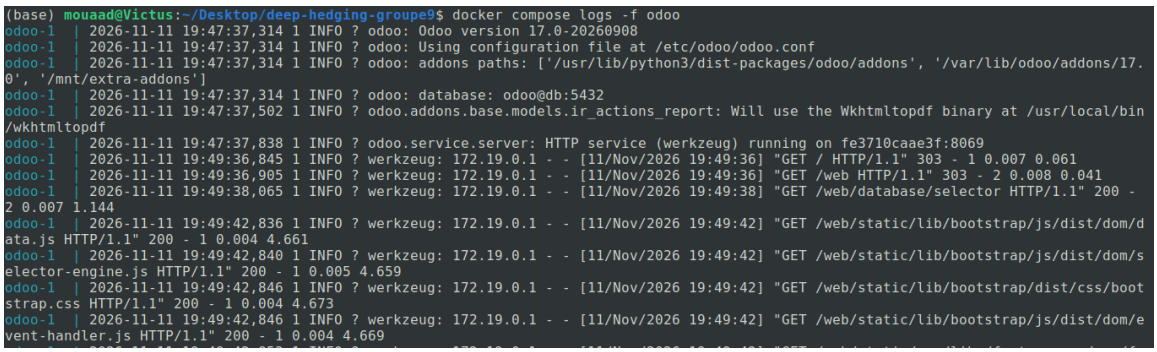


FIGURE 3 – Résultat de `docker compose logs -f odoo`.

3.7 Création de la base de données Odoo

L'accès à `http://localhost:8069` affiche la page de création de base de données. La base `odoo17` a été créée avec un compte administrateur, sans données de démonstration.

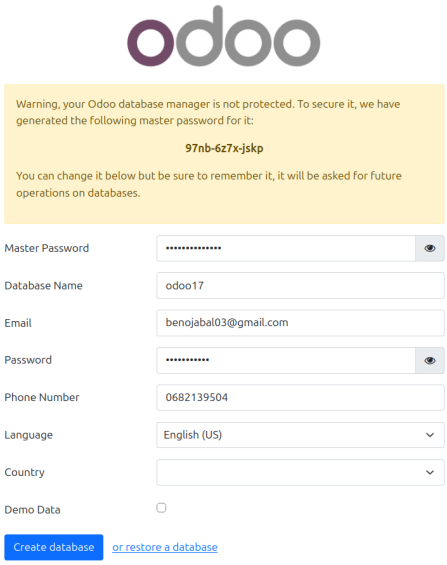


FIGURE 4 – Setup Database

3.8 Odoo opérationnel

Après la création de la base et la connexion, l'écran d'accueil d'Odoo (liste des applications) s'affiche, ce qui prouve que la pile **Docker + Odoo 17 + PostgreSQL** fonctionne correctement.

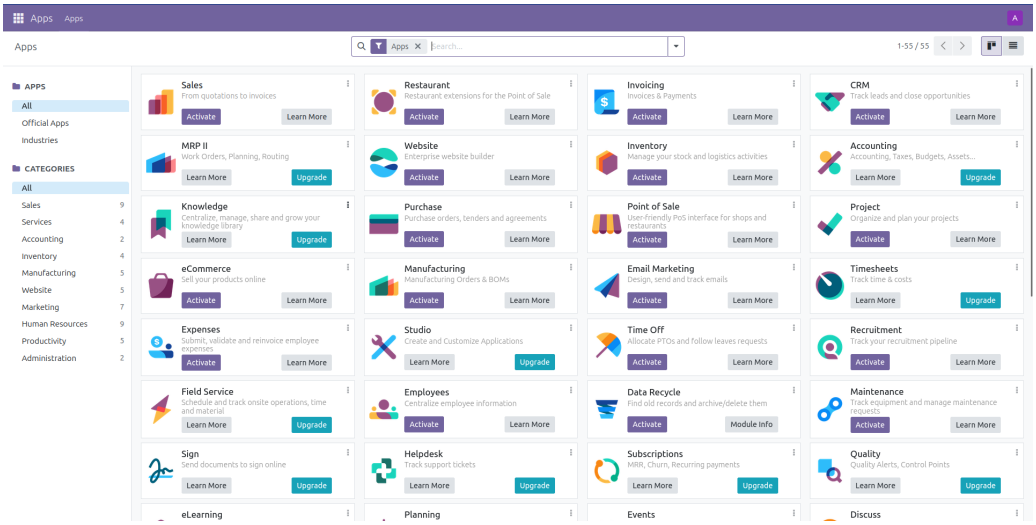


FIGURE 5 – Le dashboard global de odoo.

4 Résultats

Critère	Statut
Dépôt cloné et accessible	Fait
`docker-compose.yml` créé à la racine	Fait
Conteneurs `db` et `odoo` démarrés	Fait
Odoo accessible sur `http://localhost:8069`	Fait
Base de données `odoo17` créée	Fait
Dossier `addons/` monté dans `/mnt/extra-addons`	Fait
Capture d'écran `docs/docker_odoo.png`	Fait
Installation du module `cours_derives`	À faire (module non encore développé)

TABLE 2 – État d'avancement de la tâche S1.

5 Remarques et points d'attention

— Le module `cours_derives` n'existe pas encore : le projet démarre, et l'extension sera développée par la suite (S3 : modèle `cours.simulation.couverture`). Le dossier `addons/` est déjà prêt à le recevoir.

— Sans volumes nommés, la commande `docker compose down` supprime les conteneurs et peut entraîner la perte de la base de données. Il est conseillé d'utiliser `docker compose stop` / `start`, ou d'ajouter des volumes persistants si besoin.

— L'environnement est reproductible : tout membre du groupe peut le lancer avec un simple `docker compose up -d` après avoir cloné le dépôt.

— Le *Deep Hedging* nécessitera PyTorch ; l'inférence côté Odoo pourra demander de compléter l'image Docker (Dockerfile personnalisé) lors des semaines suivantes.

6 Livrables et commit

Fichiers ajoutés au dépôt :

— `docker-compose.yml`
— `addons/.gitkeep`
— `docs/docker_odoo.png` (et captures complémentaires)
— `docs/mini_cr_S1_mouad.tex` (ce document)

```
git add docker-compose.yml addons/.gitkeep docs/
git commit -m "[Mouad] Ajout de Docker Odoo 17 + PostgreSQL"
git push
```

7 Prochaines étapes

— Fournir l'environnement Docker au reste du groupe (README : comment lancer le projet).
— Accueillir le module `cours_derives` dans `addons/` dès qu'il est créé (Sara).
— Installer le module dans Odoo et vérifier son bon chargement (tests d'intégration).
— Préparer les tests unitaires `pytest` du simulateur, du P&L et du Delta (S2).